Practical 10

1.

AIM :

Thread-pool runner: with a fixed pool of 3 (ExecutorService), submit about 10 Runnable

tasks that print their id and sleep briefly; shut down and await completion, observing the

few threads being reused.

CODE :

```
import java.util.concurrent.*;


public class Main {

    public static void main(String[] args) throws Exception
{


        ExecutorService pool =
Executors.newFixedThreadPool(3);


        for (int i = 1; i <= 10; i++) {

            int num = i;


            pool.execute(() -> {
```

```
                System.out.println("Task " + num + " running
on " +
Thread.currentThread().getName());
            });
        }

        pool.shutdown();
        pool.awaitTermination(1, TimeUnit.MINUTES);

        System.out.println("All tasks completed");
    }
}
```

OUTPUT :



2.

AIM :

Producer–consumer: one thread adds numbers to a small shared buffer and another

removes them, coordinated with wait()/notify(); produce and consume a fixed number of

items in order with none lost.

CODE :

```
class Buffer {
    int num;
    boolean available = false;

    synchronized void produce(int n) throws
InterruptedException {
        while (available) {
            wait();
        }

        num = n;
        available = true;

        System.out.println("Produced: " + n);

        notifyAll();
    }

    synchronized void consume() throws InterruptedException {
        while (!available) {
```

```
            wait();
        }

        System.out.println("Consumed: " + num);

        available = false;

        notifyAll();
    }
}

public class ProducerConsumer {
    public static void main(String[] args) {

        Buffer b = new Buffer();

        Thread t1 = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    b.produce(i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
```

```
        });

        Thread t2 = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    b.consume();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        t1.start();
        t2.start();
    }
}
```

OUTPUT :



```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
```

3.

AIM :

(For Advanced Learners – home) Reproduce a deadlock between two threads that lock

two objects in opposite order, then remove it with a consistent lock order. Choose the

scenario yourself.

CODE :

```
public class Main {


    static Object a = new Object();
    static Object b = new Object();


    static void transfer(String name) {
        synchronized (a) {
```

```
            System.out.println(name + " locked A");

            synchronized (b) {
                System.out.println(name + " locked B");
                System.out.println(name + " transfer completed");
            }
        }
    }

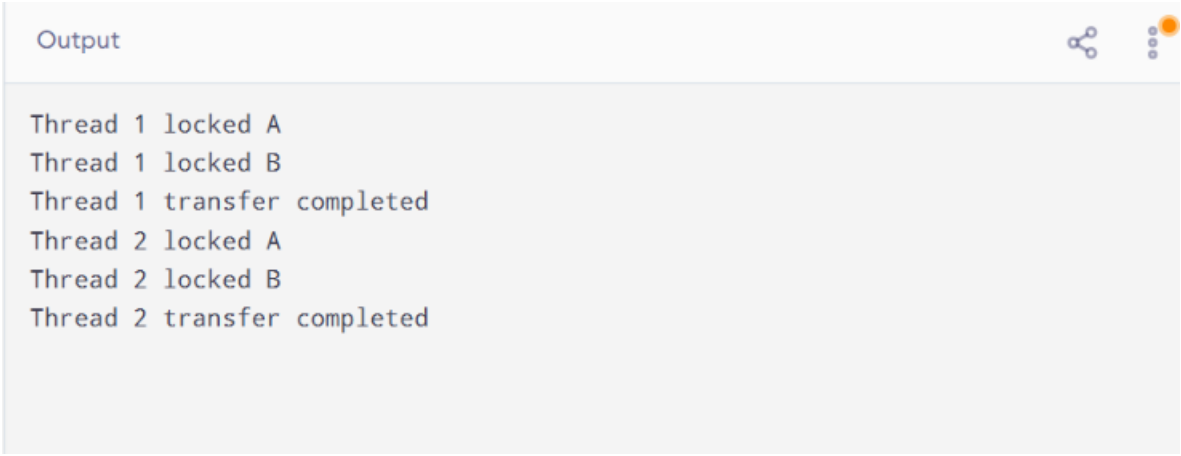
    public static void main(String[] args) {

        Thread t1 = new Thread(() -> {
            transfer("Thread 1");
        });

        Thread t2 = new Thread(() -> {
            transfer("Thread 2");
        });

        t1.start();
        t2.start();
    }
}
```

OUTPUT :



LEARNING OUTCOME :

1. Understand and implement multithreading using Java.
2. Execute multiple tasks using ExecutorService and thread pools.
3. Implement the Producer–Consumer problem using wait() and notify().
4. Understand deadlocks and prevent them using a consistent lock order.
5. Apply multithreading concepts to process banking transactions safely.